

Capitolo 2b - Progettazione e realizzazione di un SO



Scopi della progettazione

La progettazione del sistema è influenzata dalla scelta dell'architettura fisica e del tipo di sistema. I requisiti generali si possono distinguere in:

- obiettivi degli utenti:
 - deve essere utile;
 - facile da imparare e usare;
 - affidabile;
 - sicuro e veloce.
- obiettivi del sistema:
 - facile da progettare, realizzare e mantenere;
 - flessibile;
 - affidabile;
 - senza errori;
 - efficiente.

Meccanismi e politiche

Definiamo la distinzione fra:

- **Meccanismi:** *come* eseguire qualcosa.
- **Criteri o Politiche:** *che cosa* si debba fare.

Questa distinzione è importante ai fini della flessibilità. Le politiche sono soggette a cambiamenti di luogo o di tempo, nei casi peggiori il cambio di una politica può richiedere il cambiamento del meccanismo sottostante, per questo è preferibile adottare dei meccanismi generali.

Realizzazione

I primi sistemi operativi erano scritti in linguaggio assembly, ad oggi la maggior parte è scritta in linguaggi di alto livello:

- i livelli più bassi del kernel → Assembly e C;
- le routine di livello superiore → C e C++;
- libreria di sistema → C++ e linguaggi di più alto livello.

Vantaggi linguaggio ad alto livello:

- codice scritto più rapidamente, più compatto, più facile da capire e correggere;
- più facile da adattare a un'altra architettura (*porting*).

Svantaggi:

- minore velocità d'esecuzione;
- maggiore occupazione di spazio di memoria (problematica superata nei sistemi moderni).

I miglioramenti principali nelle prestazioni dei SO sono dovuti a strutture dati e algoritmi migliori. Solo una parte del codice assume particolare importanza:

- gestore degli interrupt;
- gestore dell'i/o;
- gestore della memoria;
- scheduler della cpu.

Struttura SO

Struttura monolitica

È la struttura più semplice → **assenza di struttura**. Le funzionalità del kernel vengono inserite in un singolo file binario statico che viene eseguito in un unico spazio di indirizzamento.

Svantaggi: difficile da implementare ed estendere.

Vantaggi: veloce ed efficiente, l'interfaccia alle chiamate di sistema presenta un overhead molto ridotto e la comunicazione all'interno del kernel è veloce.

Un esempio di tale struttura è il sistema operativo Unix originale, composto da due parti principali: il kernel e i programmi di sistema. Il kernel di Unix è suddiviso in interfacce e driver di dispositivi, formando una struttura parzialmente stratificata. Analogamente, il sistema operativo Linux, basato su Unix, segue una struttura simile. Le applicazioni comunicano con il kernel attraverso l'interfaccia alle chiamate di sistema, utilizzando spesso la libreria standard del linguaggio C (glibc).

Approccio stratificato

Un sistema monolitico è anche chiamato **strettamente accoppiato** (*tightly coupled*) ⇒ le modifiche a una parte del sistema possono avere effetti su altre parti.

È possibile progettare anche un sistema **debolmente accoppiato** (*loosely coupled*) = suddiviso in componenti separati più piccoli con funzionalità specifiche e limitate. Questi componenti formano il kernel.

Vantaggio: i cambiamenti in un componente influiscono solo su quel componente, consentendo maggiore libertà nella creazione e modifica dei meccanismi interni.

Per rendere modulare un SO, un modo per farlo è utilizzando un **approccio stratificato**: il sistema è suddiviso in un certo numero di livelli o strati: il più basso corrisponde all'hw (strato 0), il più alto all'interfaccia utente (strato N).

Vantaggio: semplicità di progettazione e debugging.

Gli strati sono composti in modo che ciascuno usi solo funzioni (operazioni) e servizi appartenenti a strati di livello inferiore.

I sistemi stratificati sono utilizzati nelle reti di computer (tcp/ip) e in applicazioni web.

Pochi SO utilizzano un approccio a strati puro, in quanto è difficile definire in modo appropriato le funzionalità di ogni strato. Inoltre, hanno prestazioni scarse a causa dell'overhead dovuto al fatto che la richiesta di un servizio deve attraversare più strati.

Microkernel

Si progetta il SO rimuovendo dal kernel tutti i componenti non essenziali, realizzandoli come programmi di livello utente e di sistema. Si ottiene così un kernel di dimensioni ridotte. In generale, i servizi minimi che deve offrire sono: gestione dei processi, della memoria e di comunicazione.

Vantaggi:

- facilità di estensione del SO → i nuovi servizi si aggiungono allo spazio utente e non comportano modifiche al kernel;
- se il kernel deve essere modificato, i cambiamenti da fare sono ridotti e il SO è più semplice da portare su diverse architetture;
- maggiore garanzie di sicurezza e affidabilità → i servizi si eseguono in gran parte come processi utenti e non del kernel.

Svantaggi:

- cali di prestazioni dovuti all'overhead.

Moduli

Attualmente, l'approccio migliore si basa sull'utilizzo di moduli del kernel. Il kernel deve fornire i servizi principali, mentre gli altri sono implementati in modo dinamico, quando il kernel è in esecuzione.

È più flessibile di un sistema a strati, perché ogni modulo può chiamare qualsiasi altro modulo, ed è più efficiente di un microkernel, poiché i moduli non devono invocare la funzionalità di trasmissione di messaggi per comunicare.

Linux utilizza moduli del kernel caricabili (lkm, *loadable kernel module*), per supportare i driver dei dispositivi e i file system. I LKM possono essere inseriti nel kernel all'avvio del sistema o durante l'esecuzione. Se il kernel di Linux non dispone del driver necessario, questo può essere caricato dinamicamente. I LKM possono anche essere rimossi dal kernel durante l'esecuzione.

Generazione di un SO

La generazione di un SO è necessaria quando si desidera installare un SO diverso da quello preinstallato, aggiungerne altri, o installarlo su un computer privo di SO.

Per creare un SO da zero, i passaggi fondamentali sono:

1. Scrivere/ottenere il codice sorgente del SO.
2. Configurare il SO per l'hw su cui verrà eseguito (definendo le funzionalità da includere, spesso tramite un **file di configurazione**).

3. Compilare il SO.
4. Installare il SO.
5. Avviare il computer con il nuovo SO.

Esistono tre approcci principali per configurare e generare il SO, che differiscono per livello di personalizzazione, velocità e generalità del sistema risultante:

1. **Ricompilazione completa (System Build):**

Si modifica il codice sorgente in base al file di configurazione e si ricompila completamente l'intero sistema. Massima personalizzazione, sistema conforme all'hw. Comune per sistemi embedded con configurazione statica. Più lento.

2. **Collegamento di moduli precompilati:**

La descrizione del sistema seleziona moduli oggetto precompilati (es. driver di dispositivi I/O) da una libreria, che vengono collegati per formare l'OS.

Più veloce, non richiede ricompilazione. Il sistema risultante può essere troppo generico e meno ottimizzato. Tipico dei moderni SO che usano moduli del kernel caricabili per modifiche dinamiche.

3. **Sistema completamente modulare:**

La selezione e l'inclusione dei moduli avviene al momento dell'esecuzione (runtime).

Più flessibile e facile da modificare al variare dell'hw. La generazione consiste solo nell'impostare i parametri di configurazione.

Avvio del sistema

Il processo di **boot** è la sequenza di passaggi che permette al computer di caricare e avviare il kernel del SO, rendendolo disponibile per l'uso da parte dell'hw.

Il processo si svolge generalmente in quattro fasi principali:

1. **Localizzazione del kernel:** il programma di **bootstrap** (o *boot loader*), individua l'immagine del kernel.
2. **Caricamento e avvio del kernel:** il kernel viene caricato in memoria (RAM) e avviato.
3. **Inizializzazione dell'hw:** il kernel prende il controllo e inizializza l'hw del sistema.
4. **Montaggio del File System Principale:** viene reso accessibile il file system radice (**root file system**).

Il processo di avvio può essere gestito da due tipi principali di **firmware** presenti sulla scheda madre:

1. **Processo a più fasi (basato su BIOS):**

- All'accensione, viene eseguito un piccolo boot loader nel firmware BIOS (Basic Input Output System).
- Questo BIOS carica un secondo, più complesso, boot loader che si trova in un'area fissa del disco chiamata **blocco di avvio (boot block)**.
- Il boot loader del boot block carica il SO.

2. **Processo Unificato (basato su UEFI):**

- I sistemi moderni utilizzano UEFI (Unified Extensible Firmware Interface), che sostituisce il BIOS.
- UEFI è un gestore di avvio unificato e completo, risultando più veloce del processo a più stadi del BIOS e offrendo un miglior supporto per hw moderno.

Il boot loader (BIOS/boot block o UEFI) esegue diverse attività prima di avviare il SO:

- esegue la **diagnostica** per verificare lo stato dell'hw (memoria, CPU, dispositivi);
- **inizializza** tutti gli aspetti del sistema (registri della cpu, controller dei dispositivi);
- carica il file contenente il kernel in memoria.

Debugging dei SO

Il debugging è l'attività di **individuare e risolvere (bug)** hw e sw in un sistema. Include anche l'attività di **performance tuning** (regolazione delle prestazioni) per eliminare i colli di bottiglia del sistema.

Analisi dei malfunzionamenti (Processi vs Kernel)

Il debugging si concentra principalmente su due livelli: a livello di **processo utente** e a livello di **kernel**.

Debugging a livello di processo utente

Quando un processo utente fallisce:

- Il SO registra le informazioni sull'errore in un **file di log**.
- Viene acquisita e salvata un'immagine del contenuto della memoria utilizzata dal processo, **core dump**.
- Uno strumento chiamato **debugger** può esaminare i programmi in esecuzione o analizzare il core dump per aiutare il programmatore a identificare l'errore.

Debugging a livello di kernel

Il debugging a livello di kernel è molto più difficile a causa della sua dimensione, complessità, e del suo controllo diretto sull'hw.

- Un guasto nel kernel è chiamato **crash**.
- In caso di crash, le informazioni sull'errore vengono salvate in un **file di log** e lo stato della memoria in un'immagine chiamata **crash dump**.

Tecniche per il debugging del kernel

A causa della natura critica e complessa del kernel (un malfunzionamento nel file system impedirebbe di salvare lo stato su file):

- **Salvataggio su area dedicata del disco:** per salvare lo stato di memoria del kernel, si utilizza una sezione del disco adibita esclusivamente a questo scopo, situata al di fuori del file system principale.
- **Procedura di salvataggio:** quando il kernel rileva un errore irrecoverabile, scrive l'intero contenuto della memoria (o le parti rilevanti) in quest'area dedicata.
- **Analisi post-riavvio:** al riavvio del kernel, viene eseguito un processo che raccoglie i dati da quest'area del disco e li scrive in un file apposito all'interno del file system per la successiva analisi.

Tracing

Il tracing è una tecnica di monitoraggio utilizzata per la regolazione delle prestazioni (**performance tuning**), volta a identificare ed eliminare i **colli di bottiglia** del sistema. A differenza dei contatori che forniscono solo valori statistici correnti, il tracing raccoglie dati dettagliati relativi a eventi specifici.

Gli strumenti di tracing registrano i passaggi completi coinvolti in un evento, come ad esempio tutte le fasi necessarie per l'invocazione di una chiamata di sistema da parte di un processo.

Esempi in Linux

Il tracing può essere effettuato sia a livello di singolo processo che a livello di sistema:

Livello	Strumento	Funzione Principale
A Livello di Processo	strace	Traccia e registra tutte le chiamate di sistema invocate da un processo.
A Livello di Processo	gdb	Un debugger a livello di codice sorgente, che consente il tracciamento dettagliato dell'esecuzione del codice.
A Livello di Sistema	perf	Una raccolta completa di strumenti per l'analisi delle prestazioni e il tracciamento degli eventi a livello di kernel e CPU.
A Livello di Sistema	tcpdump	Strumento per l' intercettazione dei pacchetti di rete , utile per tracciare il traffico e le comunicazioni.

Innovazioni e Tracing Dinamico

Il tracing e l'analisi delle prestazioni sono aree in continua evoluzione.

- Una nuova generazione di strumenti, come il bcc (BPF Compiler Collection), consente il **tracing dinamico del kernel** in Linux.
- Questa capacità migliora significativamente il modo in cui i sistemi operativi possono essere compresi, sottoposti a debug e ottimizzati in tempo reale.

BCC per il tracing dinamico

BCC è un toolkit avanzato progettato per il tracing dinamico, sicuro e a basso impatto sui sistemi Linux. Il suo scopo principale è semplificare il debugging delle interazioni tra codice utente e codice kernel, specialmente in aree del sistema non pensate per il debug, senza compromettere l'affidabilità o influenzare significativamente le prestazioni.

BCC funge da interfaccia front-end per la tecnologia eBPF (Extended Berkeley Packet Filter).

- BPF originale: sviluppato per filtrare il traffico di rete.
- eBPF: ha esteso BPF, consentendo di scrivere programmi (in un sottoinsieme del linguaggio C) che vengono compilati in istruzioni eBPF.

- **Inserimento dinamico:** queste istruzioni eBPF possono essere inserite dinamicamente in un kernel Linux in esecuzione per acquisire eventi specifici (es. chiamate di sistema) o monitorare le prestazioni (es. latenza I/O su disco).
- **Verifica di sicurezza:** prima dell'inserimento, un programma verificatore controlla le istruzioni eBPF per garantire che non influenzino negativamente la sicurezza o le prestazioni del sistema in esecuzione.